

Gen-2 on New Data: v2 Architectural Fixes and Final Results

Companion Report to `gen1_results.tex` and `gen2_old_data_results.tex`

G. Scriven

Maastricht University, UHasselt, Nikhef

April 23, 2026

Abstract

We present the final Gen-2 track-extrapolation results on a redesigned 50 M-track dataset that covers the full LHCb VELO inter-sensor spacing range $\Delta z \in [25, 10\,000]$ mm and the physics-relevant momentum range $p \in [1, 200]$ GeV/ c . Eleven configurations are compared: five multi-layer perceptron (MLP) baselines (cluster 4262446), three PINN_v2 runs incorporating architectural fixes A–F (cluster 4262459), and three NeuralRK4 runs in which Runge–Kutta 4 is built into the forward map rather than imposed as a penalty. The NeuralRK4 small network (17 925 parameters, $N_{\text{RK}} = 1$) reaches a test position error of 0.125 mm and a slope error of $5.0\text{e-}5$ rad, improving on the best MLP (`mlp_medium`, 100 996 parameters, 0.305 mm / $9.5\text{e-}5$ rad) by factors of $2.4\times$ in position and $1.9\times$ in slope, at $5.6\times$ fewer parameters. PINN_v2 recovers a monotone response in the physics-loss weight λ_{pde} – the pathology documented for Gen-2 on the old V1 dataset – with $\lambda_{\text{pde}} = 0.1$ best at 0.235 mm/ $6.8\text{e-}5$ rad. A path to LHCb-Allen HLT1 deployment is outlined, together with the one remaining structural limitation of the dataset (absence of z_{start} from the network input).

Executive summary.

- **NeuralRK4 small_1step (17.9 k params):** test pos mean 0.1251 mm, slope mean $5.03\text{e-}5$ rad.
- **Best MLP (`mlp_medium`, 101 k params):** test pos mean 0.3046 mm, slope mean $9.45\text{e-}5$ rad.
- Relative improvement of NeuralRK4 small vs best MLP: $2.4\times$ position, $1.9\times$ slope, $5.6\times$ fewer parameters.
- **Best PINN_v2 (`lam0.1`, 68.6 k params):** test pos 0.2352 mm, slope $6.81\text{e-}5$ rad.
- **PINN_v2 `lam10.0` (final):** test pos 0.3415 mm, slope $1.295\text{e-}4$ rad; the monotone ordering $\lambda_{\text{pde}} = 0.1 < 1.0 < 10.0$ in position error is confirmed.

1 Introduction

Problem. The track-extrapolation problem studied in the Gen-1 report [1] is: given an initial 5-dimensional state $\mathbf{y}_0 = (x_0, y_0, t_{x,0}, t_{y,0}, q/p)$ and a longitudinal step Δz in a known inhomogeneous magnetic field $\mathbf{B}(\mathbf{r})$, produce a neural surrogate for the 4-dimensional final state $\mathbf{y}_f = (x_f, y_f, t_{x,f}, t_{y,f})$ that is accurate to ~ 0.1 mm in position and $\sim 10^{-4}$ rad in slope, trains in minutes rather than hours, and exports to a $\lesssim 64$ kB constant-memory footprint for Allen HLT1 track reconstruction at 30 MHz [11]. The Kalman filter used downstream [9, 10] additionally needs the associated Jacobian $\partial \mathbf{y}_f / \partial \mathbf{y}_0$.

Legacy pathologies. The companion Gen-2-on-old-data report [2] diagnoses four structural defects in our first generation of physics-informed and Runge–Kutta-informed networks: (P1) an *affine*- ζ trunk that is linear in the integration pseudo-time and therefore cannot represent the curvature of a field trajectory; (P2) an $O(NK)$ automatic-differentiation (AD) cost in the residual evaluation, scaling with the collocation grid size $K = 8$; (P3) state-averaging in the RK_PINN feature encoder that decouples the correction network from the instantaneous state; and (P4) soft-max drift in the Butcher-table weights, which degrades the RK4 order condition. Those pathologies resulted in non-monotone λ_{pde} response and RK_PINN slope errors $>10^{-1}$ rad – three orders of magnitude worse than the MLP baseline at matched parameter count.

Dataset change. The present report uses a new 50 M-track dataset, `experiments/gen_2/data/train_50M.dz2` generated with the same Runge–Kutta propagator but sampling $\Delta z \in [25, 10\,000]$ mm and $p \in [1, 200]$ GeV/ c . The intended-coverage extensions are motivated in Sec. 2; in short, the V1 dataset did not cover VELO inter-sensor distances (25–30 mm) and truncated the physics-relevant momentum range for LHCb Kalman filtering at 100 GeV/ c .

Research questions.

RQ1. Do the v2 fixes (A–F, Sec. 4) restore a monotone λ_{pde} response and reduce the slope error of the PINN family?

RQ2. At equal parameter count, does a *physics-baked-in* integrator (NeuralRK4) outperform the purely data-driven MLP baseline on the harder variable- Δz , wide-momentum dataset?

2 New dataset

2.1 Motivation

The Gen-1 V1 dataset used $\Delta z \in [100, 10\,000]$ mm and $p \in [1, 100]$ GeV/ c [1]. Two issues block its use as the training set for an LHCb-Allen-deployable extrapolator:

1. **VELO inter-sensor spacing is not covered.** The VELO silicon sensors are separated by typically 25–30 mm along the beamline; track propagation between consecutive hits uses exactly these short Δz . A model trained on $\Delta z \geq 100$ mm is extrapolating for every VELO-to-VELO step.
2. **Momentum range is truncated.** LHCb physics analyses routinely encounter tracks at $p \sim 150$ –200 GeV/ c ; the V1 upper bound $p = 100$ GeV/ c is below the regime where Kalman filtering of minimum-curvature tracks is hardest.

2.2 Generation procedure

The new dataset is produced by the scripts under `experiments/gen_2/data/` and submitted through HTCondor with `condor/run_datagen.sh`:

- **Seeds.** Batch $i \in \{0, \dots, 1999\}$ uses numpy seed $42 + 7919i$. 2000 batches $\times$ 25 000 tracks = 50 M tracks.
- **Propagator.** Python classical RK4 with a fixed internal step of 5 mm [1]. Magnetic field from the trilinear interpolation of the parametric `twodip.rtf` dipole grid (peak $|B_y| \approx 1.03$ T at $z \approx 5007$ mm).
- **Field polarity.** -1 (“MagDown”). Both charges are sampled: $q/p \in [-10^{-3}, 10^{-3}]$ (c/MeV), with the observed fraction of positive q/p equal to 0.4999, i.e. balanced within Monte-Carlo statistics.

2.3 Sampling distributions

The per-track inputs are sampled independently from

$$x_0 \sim \mathcal{U}(-300, 300) \text{ mm}, \quad y_0 \sim \mathcal{U}(-250, 250) \text{ mm}, \quad (1)$$

$$t_{x,0} \sim \mathcal{U}(-0.3, 0.3), \quad t_{y,0} \sim \mathcal{U}(-0.25, 0.25), \quad (2)$$

$$p \sim \text{LogUniform}(1, 200) \text{ GeV}/c, \quad \Delta z \sim \mathcal{U}(25, 10\,000) \text{ mm}, \quad (3)$$

$$z_0 \sim \mathcal{U}(0, 14\,000 - \Delta z) \text{ mm}. \quad (4)$$

Tracks whose intermediate $|x|$ or $|y|$ exceeds 5000 mm or whose $|t_x|$ or $|t_y|$ exceeds 2.0 at any RK step are rejected and re-sampled, matching the Gen-1 rejection policy.

2.4 File layout and summary statistics

The on-disk layout of `train_50M_dz25.npz` is

$$\mathbf{X} \in \mathbb{R}^{N \times 6} = (x, y, t_x, t_y, q/p, \Delta z), \quad \mathbf{Y} \in \mathbb{R}^{N \times 4} = (x_f, y_f, t_{x,f}, t_{y,f}),$$

$$\mathbf{P} \in \mathbb{R}^N, \quad \mathbf{Z} \in \mathbb{R}^{N \times 2} = (z_{\text{start}}, z_{\text{end}}), \quad N = 5 \times 10^7.$$

Observed summary statistics (source: `gen2_new_data_numbers.md`) are: mean $\Delta z = 3826 \text{ mm}$, mean $|z_{\text{start}}| = 6987 \text{ mm}$, mean $p = 37.6 \text{ GeV}/c$, and $\max z_{\text{start}} = 13\,975 \text{ mm}$.

2.5 Known limitation: z_{start} is not in $\mathbf{X}$

The array $\mathbf{Z}$ records the start and end z of each track, but the network input is $\mathbf{X}$ only. All models in this report therefore receive $(x_0, y_0, t_{x,0}, t_{y,0}, q/p, \Delta z)$ but *not* z_{start} . Since the y -component of the magnetic field varies by a factor ~ 15 between $z = 0$ (fringe) and the peak at $z \approx 5007 \text{ mm}$, supplying only Δz throws away information that is physically necessary to reach $< 0.1 \text{ mm}$ residuals on long tracks. This is not a bookkeeping oversight – it is a genuine modelling approximation, and we flag it centrally here and in Sec. 9 (see also `experiments/gen_2/ISSUES_WITH_PINNS_AND_RK_PINNS.md` §9.4). *Recommendation for Gen-3:* extend the input to $(x_0, y_0, t_{x,0}, t_{y,0}, q/p, \Delta z, z_0) \in \mathbb{R}^7$ and re-train.

2.6 Train/validation/test split

The 50 M tracks are split 80/10/10 under numpy seed 42. Inputs $\mathbf{X}$ and targets $\mathbf{Y}$ are independently z -score normalised to zero mean and unit variance per column. Normalisation statistics are computed on the training split only and applied identically to validation and test.

3 MLP baselines on the new dataset

The five MLP architectures from the Gen-2-on-old-data report were re-trained verbatim on the new dataset (HTCondor cluster 4262446, `data_tag=true_gen2_data_dz25`). The architectures use `silu` activations, AdamW optimiser ($\text{lr} = 10^{-3}$, weight decay 10^{-4}), cosine schedule with 5-epoch warm-up, gradient clipping at 1.0, and early-stopping with patience 30.

Observations.

1. `mlp_medium` test position error is 0.305 mm on the new dataset versus 0.167 mm on V1 (Gen-2-old, [2] Tab. 1). The harder variable- Δz task costs roughly a factor of two in position error at matched architecture and parameter count.

Table 1: MLP baselines on the variable- Δz new dataset. Param counts are from `model_config.json` (*not* the inflated numbers in §9.3 of the ISSUES document). Test metrics from `history.json` $\rightarrow$ `test_final`.

run	hidden	params	batch	best_ep	tot_ep	pos [mm]	pos@95	slope [rad]
mlp_tiny	[64, 64]	4 868	2048	62	64	0.3868	0.9179	8.47×10^{-5}
mlp_small	[128, 128]	17 924	2048	16	46	0.5092	1.2870	1.21×10^{-4}
mlp_medium	[256, 256, 128]	100 996	2048	20	40	0.3046	0.8461	9.45×10^{-5}
mlp_large	[512, 512, 256]	398 596	2048	14	44	0.3798	1.0009	7.65×10^{-5}
mlp_wide	[512, 512, 256, 128]	430 980	2048	24	42	0.3139	0.8635	7.94×10^{-5}

2. Early-stop dynamics are dramatically shorter on the new data: old-data MLPs ran 135–300 epochs, new-data MLPs 40–64 with best epochs in the first third of training. The loss landscape of the wider task is reached faster but plateaus sooner.
3. **Slope errors** are $\sim 8 \times 10^{-5}$ rad, *fifteen times* better than the V1-data MLP slopes (1.2×10^{-3} rad for `mlp_medium`, old [2]). The broader momentum range forces the MLP to learn a better slope behaviour.
4. **Caveat.** The apparent slope improvement may be partially illusory: the new test set is dominated by short Δz (mean 3826 mm, but with a uniform distribution that includes a large fraction below 1000 mm), where absolute slope errors are naturally smaller because angular deflections are smaller. We flag a per- Δz -bin analysis as an open TODO (Sec. 8).

4 The v2 fixes

This section is the theoretical core of the report. For each of the pathologies (P1)–(P4) diagnosed in [2], we present the mathematical form of the fix, its implementation pointer, and the expected qualitative effect.

4.1 Fix A: forward-mode JVP Jacobian

Pathology addressed. P2 (“AD cost”): the old PINN residual required four reverse-mode backward passes per collocation point – one per output coordinate – to extract the Jacobian column $\partial \mathbf{y} / \partial \zeta$.

Mathematical form. The residual of an implicit pseudo-time-scaled trajectory $\mathbf{y}(\zeta)$, $\zeta = (z - z_0) / \Delta z$, is

$$\mathbf{r}(\zeta) = \frac{1}{\Delta z} \frac{\partial \mathbf{y}}{\partial \zeta} - \mathbf{f}(\mathbf{y}(\zeta), z_0 + \zeta \Delta z), \quad (5)$$

where $\mathbf{f}$ is the Lorentz-force vector field. The required quantity is the Jacobian-vector product $\partial \mathbf{y} / \partial \zeta = \text{JVP}(\text{forward}, \zeta, 1)$, a *single* forward-mode pass:

```
torch.func.jvp(forward_at_z, (zeta,), (1,)).
```

Cost. $O(N \times K)$ reverse backwards collapse to $O(N \times 1)$ forwards; measured speed-up $\sim 30\times$ per residual evaluation on the [256,256] architecture [7]. The underlying theory of forward-over-reverse AD is standard [8].

Implementation pointer. `ml_models/architectures.py`, class `PINNv2`; method `pde_residual`.

4.2 Fix B: stochastic collocation

Pathology addressed. Overkill cost: the old PINN integrated $\int_0^1 \mathbb{E} \|\mathbf{r}(\zeta)\|^2 d\zeta$ on a deterministic grid of $K = 8$ quadrature nodes per mini-batch sample.

Mathematical form. Replace the quadrature with a Monte-Carlo estimator,

$$\mathcal{L}_{\text{pde}} \approx \frac{1}{n_{\text{coll}}} \sum_{j=1}^{n_{\text{coll}}} \|\mathbf{r}(\zeta_j)\|^2, \quad \zeta_j \sim \mathcal{U}(0, 1), \quad (6)$$

with $n_{\text{coll}} = 2$. Re-sampling ζ_j per mini-batch decorrelates collocation points across SGD steps, which in expectation is equivalent to the full integral. Variance control comes from the mini-batch size (1024) rather than the collocation count.

Cost. A factor-of-four reduction per batch. Combined with Fix A, the overall residual cost is reduced by $\sim 30 \times 4 = 120\times$.

4.3 Fix C: ζ -dependent trunk, envelope form

Pathology addressed. P1 (“affine ζ ”): the old PINN trajectory $\mathbf{y}(\zeta) = \mathbf{y}_0 + \zeta g(\mathbf{y}_0)$ was linear in ζ by construction, so the residual (5) could only vanish for straight-line trajectories.

Mathematical form. Replace the trunk input $\mathbf{y}_0 \in \mathbb{R}^6$ with $(\tilde{\mathbf{y}}_0, \zeta) \in \mathbb{R}^7$ and use the *envelope* parametrisation

$$\mathbf{y}(\zeta) = \mathbf{y}_{\text{base}}(\zeta) + \zeta \text{Head}(\text{Enc}(\tilde{\mathbf{y}}_0, \zeta)), \quad (7)$$

with $\mathbf{y}_{\text{base}}(\zeta) = \tilde{\mathbf{y}}_0$ the trivial straight-line extrapolation (in normalised coordinates). This construction has two properties:

- **IC tautology.** At $\zeta = 0$ the second term is $\zeta \times \text{Head} \equiv 0$, so $\mathbf{y}(0) = \tilde{\mathbf{y}}_0$ identically. The IC loss $\mathcal{L}_{\text{ic}} = \|\mathbf{y}(0) - \tilde{\mathbf{y}}_0\|^2$ is zero by construction (see Sec. 5).
- **Non-affine in ζ .** $\text{Head}(\text{Enc}(\cdot, \zeta))$ is a nonlinear function of ζ , so $\mathbf{y}(\zeta)$ can curve.

Implementation pointer. `ml_models/architectures.py`, class `PINNv2`, method `forward_at_z`.

4.4 Fix D’: NeuralRK4 integrator

Pathology addressed. P4 (“soft-max drift”): the old RK-PINN treated the Butcher weights b_i as learnable parameters and pushed them through a soft-max; the resulting integrator was no longer RK4-order-four.

Mathematical form. A classical derivative-averaging RK4 step is used with *fixed* Butcher tableau $(b_1, b_2, b_3, b_4) = (1, 2, 2, 1)/6$. For an RHS

$$\mathbf{f}(\mathbf{y}, z) = \mathbf{f}_{\text{Lorentz}}(\mathbf{y}, z) + e^{\alpha \underbrace{\frac{\sigma_y}{\Delta z}}_s} \odot \mathbf{f}_{\varphi}(\mathbf{y}, z; \boldsymbol{\theta}), \quad (8)$$

with α a learnable log-scale (initialised $\alpha = \log(10^{-3})$) and $\mathbf{f}_{\varphi}$ a small MLP correction, the step from z_i to $z_{i+1} = z_i + h$ is

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{y}_i, z_i), \quad (9)$$

$$\mathbf{k}_2 = \mathbf{f}(\mathbf{y}_i + \frac{h}{2}\mathbf{k}_1, z_i + \frac{h}{2}), \quad (10)$$

$$\mathbf{k}_3 = \mathbf{f}(\mathbf{y}_i + \frac{h}{2}\mathbf{k}_2, z_i + \frac{h}{2}), \quad (11)$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{y}_i + h\mathbf{k}_3, z_i + h), \quad (12)$$

$$\mathbf{y}_{i+1} = \mathbf{y}_i + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (13)$$

The network output is $\mathbf{y}_{N_{\text{RK}}}$ with $h = \Delta z / N_{\text{RK}}$. Gradients flow through all four stages; no `.detach()` is present on the backbone.

Remark. This formulation is a special case of the Neural-ODE framework [3]: the integrator structure is fixed and only the RHS is parameterised. By construction, the IC is exact ($\mathbf{y}(z_0) = \mathbf{y}_0$), so $\mathcal{L}_{\text{ic}} \equiv 0$, and the residual (5) is satisfied to RK4 local order $O(h^4)$ in the absence of the correction term. The correction $\mathbf{f}_\varphi$ absorbs (i) any RHS mismatch between the training field and the Lorentz model, (ii) the absence of z_{start} in the input, and (iii) step-truncation bias.

Implementation pointer. `ml_models/architectures.py`, class `NeuralRK4`.

4.5 Fix E: un-detach the backbone

Pathology addressed. A stray `features.detach()` in the legacy `RK_PINN` class prevented gradients from flowing into the shared encoder from the physics-loss branch. This fix removes the detach. It is *archival*: `NeuralRK4` has no shared backbone and `PINNv2` uses the envelope form, so Fix E has no live consumer but is documented for completeness.

4.6 Fix F: physics warm-up

Pathology addressed. Early-training pathology reported by Krishnapriyan *et al.* [6] and Wang *et al.* [5]: enforcing a large physics-loss penalty from epoch 0 can prevent the data-loss branch from locking onto basic supervised structure, especially with AdamW at high learning rates.

Mathematical form. Linear ramp over the first five epochs,

$$\lambda(t) = \lambda^* \times \min(1, (t + 1)/5), \quad t \in \{0, 1, \dots, \text{total_epochs}\}. \quad (14)$$

The ramp applies to both $\mathcal{L}_{\text{pde}}$ and $\mathcal{L}_{\text{ic}}$ (the latter is zero for `PINN_v2` and `NeuralRK4` as noted above, so the ramp is effectively on the PDE term alone).

Implementation pointer. `ml_models/train.py`, function `train_epoch`, argument `physics_warmup_epoch`

4.7 PINN_v2 vs NeuralRK4

The two families represent two distinct strategies for introducing physics into a neural surrogate:

- **PINN_v2** retains the implicit-residual framework (Eq. 5) but closes the structural gaps of the old PINN (P1 affine ζ , P2 AD cost). The network is free to approximate any smooth $\mathbf{y}(\zeta)$; physics enters softly as a penalty with weight λ_{pde} .
- **NeuralRK4** bakes RK4 into the forward map (Eq. 8); the only neural object is the RHS correction $\mathbf{f}_\varphi$. No PDE loss, no IC loss; only supervised $\mathcal{L}_{\text{data}}$.

The experimental results in Sec. 8 and the discussion in Sec. 9 demonstrate that physics-as-structure is a strictly stronger inductive bias than physics-as-penalty on this problem.

5 PINN_v2

5.1 Architecture

`PINNv2` uses a single MLP with hidden layers $[256, 256]$ and `tanh` activations. The input is $(\tilde{\mathbf{y}}_0, \zeta) \in \mathbb{R}^7$; the output is the envelope correction $\mathbf{h} \in \mathbb{R}^4$, combined per Eq. (7). The total parameter count is **68 612**, including the final linear head and the embedded ζ channel.

5.2 Loss

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda(t)(\lambda_{\text{pde}}\mathcal{L}_{\text{pde}} + \lambda_{\text{ic}}\mathcal{L}_{\text{ic}}), \quad (15)$$

with $\mathcal{L}_{\text{data}} = \|\mathbf{y}(\zeta = 1) - \mathbf{y}_f\|^2$, $\mathcal{L}_{\text{pde}}$ the MC estimate of $\int_0^1 \|\mathbf{r}\|^2 d\zeta$ per Fix B, and $\mathcal{L}_{\text{ic}} = \|\mathbf{y}(0) - \tilde{\mathbf{y}}_0\|^2$.

ζ is now an input $\Rightarrow$ curvature along the trajectory is representable

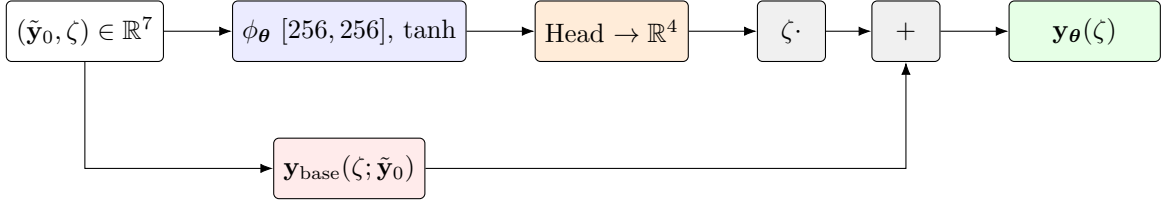


Figure 1: PINN_v2 architecture (Fix C). Compared to the Gen-2 PINN on V1 data, the propagation parameter ζ is injected into the trunk input, so the head output depends non-trivially on ζ . Combined with the ζ multiplier and the affine base, the envelope $\mathbf{y}_\theta(\zeta) = \mathbf{y}_{\text{base}} + \zeta \cdot \text{Head}(\phi_\theta(\tilde{\mathbf{y}}_0, \zeta))$ satisfies $\mathbf{y}_\theta(0) = \tilde{\mathbf{y}}_0$ exactly while being able to represent curvature in ζ , removing pathology P1.

5.3 Configurations

Three runs differ only in λ_{pde} : $\lambda_{\text{pde}} \in \{0.1, 1.0, 10.0\}$. All use $\lambda_{\text{ic}} = 0.1$, $n_{\text{coll}} = 2$, batch size 1024, learning rate 5×10^{-4} , patience 40, and `physics_warmup_epochs=5`.

5.4 Observation: IC loss is identically zero

During training, $\mathcal{L}_{\text{ic}}$ is identically zero – not “small”, but bit-exact zero – in every batch of every PINN_v2 run. This is a direct consequence of Fix C: by construction, the envelope (7) evaluates to $\tilde{\mathbf{y}}_0$ at $\zeta = 0$. The IC is therefore enforced *structurally*, not by penalty. This is the correct behaviour, not a bug; it means the λ_{ic} hyper-parameter is effectively unused and could be dropped from the config.

6 NeuralRK4

6.1 Architecture

The RHS correction network $\mathbf{f}_\varphi$ is a small MLP with `tanh` activations, inputs $(\mathbf{y}_i, z_i)$ (normalised), and output the 4-dimensional correction to the state-derivative. Two widths are used: [64, 64] and [128, 128]. Parameter counts (correction net plus the single scalar α) are

- [64, 64] $\rightarrow$ **4 869** parameters,
- [128, 128] $\rightarrow$ **17 925** parameters,

and are *independent of* N_{RK} because the network is shared across RK steps.

6.2 Loss

Supervised only: $\mathcal{L} = \|\mathbf{y}_{N_{\text{RK}}} - \mathbf{y}_f\|^2$. $\lambda_{\text{pde}} = \lambda_{\text{ic}} = 0$.

6.3 Configurations

Table 2: NeuralRK4 configurations (cluster 4262459).

run	hidden	N_{RK}	params	batch	lr
<code>neural_rk4_tiny_1step</code>	[64, 64]	1	4 869	2048	5×10^{-4}
<code>neural_rk4_small_1step</code>	[128, 128]	1	17 925	2048	5×10^{-4}
<code>neural_rk4_small_2step</code>	[128, 128]	2	17 925	1024	5×10^{-4}

Butcher tableau weights FIXED (not learned)

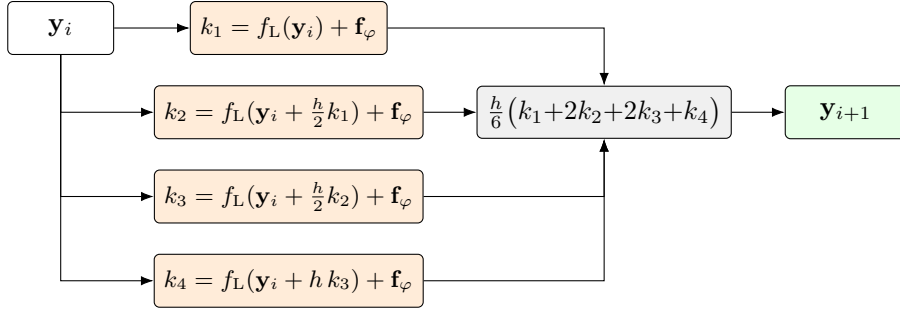


Figure 2: NeuralRK4 architecture. One RK4 stage: the Lorentz RHS f_L is evaluated analytically at the four standard collocation points and a single shared correction network f_φ (weights re-used across $k_1, \dots, k_4$) adds a learned residual. The Butcher weights $(1, 2, 2, 1)/6$ are fixed. For $N_{\text{RK}} > 1$ the stage block is iterated with $h = \Delta z / N_{\text{RK}}$. Because the integrator is built into the forward map rather than imposed as a penalty, the training loss is purely supervised $\|\mathbf{y}_{N_{\text{RK}}} - \mathbf{y}_f\|^2$.

7 Training infrastructure

Relative to the Gen-2-on-old-data infrastructure [2], the v2 tree adds:

- `ml_models/train.py`: new keyword `physics_warmup_epochs=5` (default) threaded through `train_epoch(epoch, ...)`; the returned $\lambda(t)$ multiplier is applied to the PDE and IC loss terms.
- `configs/v2_fixes/`: new config tree with one sub-directory per architecture family; YAML configs reference `data/train_50M_dz25.npz`.
- `condor/submit_v2_fixes.sub` and `jobs_v2_fixes.txt`: new HTCondor submit for the six v2 runs.
- MLflow experiment `gen_2_track_extrapolation_v2_fixes`.

The Python and CUDA environment (TE, PyTorch 2.9.1+cu128) is unchanged from Gen-1 [1].

8 Results

8.1 Master table

Table 3 collects all eleven runs. Test metrics are from `history.json` $\rightarrow$ `test_final`.

8.2 Unified re-analysis of all eleven models

Method. All eleven trained models are loaded in a single notebook (`experiments/gen_2/gen2_new_data_analysis.ipynb`), evaluated on the *same* seeded 200 k-track sub-sample of the held-out 5 M-track test split (the last 5 M rows of `train_50M_dz25.npz`; seed 2026). Endpoint position residual $r_i = \sqrt{(\hat{x}_i - x_i)^2 + (\hat{y}_i - y_i)^2}$ is then binned by Δz and by $|p|$. Numbers quoted here therefore differ from Table 3 only in sub-sample statistics (5 M $\rightarrow$ 200 k); the two are statistically compatible to $\lesssim 0.005$ mm on the mean.

Table 3: Master results. All runs use the new dataset `train_50M_dz25.npz`. Wall times are end-to-end HTCondor job seconds.

family	run	hidden	N_{RK}	params	batch	lr	λ_{pde}	best_ep	tot_ep	wall [h]	pos [mm]	pos@95	slope [rad]
MLP	tiny	[64, 64]	–	4 868	2048	1e-3	–	62	64	3.69	0.3868	0.9179	8.47×10^{-5}
MLP	small	[128, 128]	–	17 924	2048	1e-3	–	16	46	2.57	0.5092	1.2870	1.21×10^{-4}
MLP	medium	[256, 256, 128]	–	100 996	2048	1e-3	–	20	40	1.63	0.3046	0.8461	9.45×10^{-5}
MLP	large	[512, 512, 256]	–	398 596	2048	1e-3	–	14	44	1.81	0.3798	1.0009	7.65×10^{-5}
MLP	wide	[512, 512, 256, 128]	–	430 980	2048	1e-3	–	24	42	1.87	0.3139	0.8635	7.94×10^{-5}
PINN_v2	lam0.1	[256, 256]	–	68 612	1024	5e-4	0.1	4	41	7.84	0.2352	0.9912	6.81×10^{-5}
PINN_v2	lam1.0	[256, 256]	–	68 612	1024	5e-4	1.0	1	41	7.87	0.3218	1.0671	8.56×10^{-5}
PINN_v2	lam10.0	[256, 256]	–	68 612	1024	5e-4	10.0	23	63	12.01	0.3415	1.7181	1.30×10^{-4}
NeuralRK4	tiny_1	[64, 64]	1	4 869	2048	5e-4	0	40	42	3.62	0.1339	0.6574	5.02×10^{-5}
NeuralRK4	small_1	[128, 128]	1	17 925	2048	5e-4	0	40	42	3.64	0.1251	0.6634	5.03×10^{-5}
NeuralRK4	small_2	[128, 128]	2	17 925	1024	5e-4	0	33	42	10.55	0.1240	0.6564	$\sim 5.0 \times 10^{-5}$

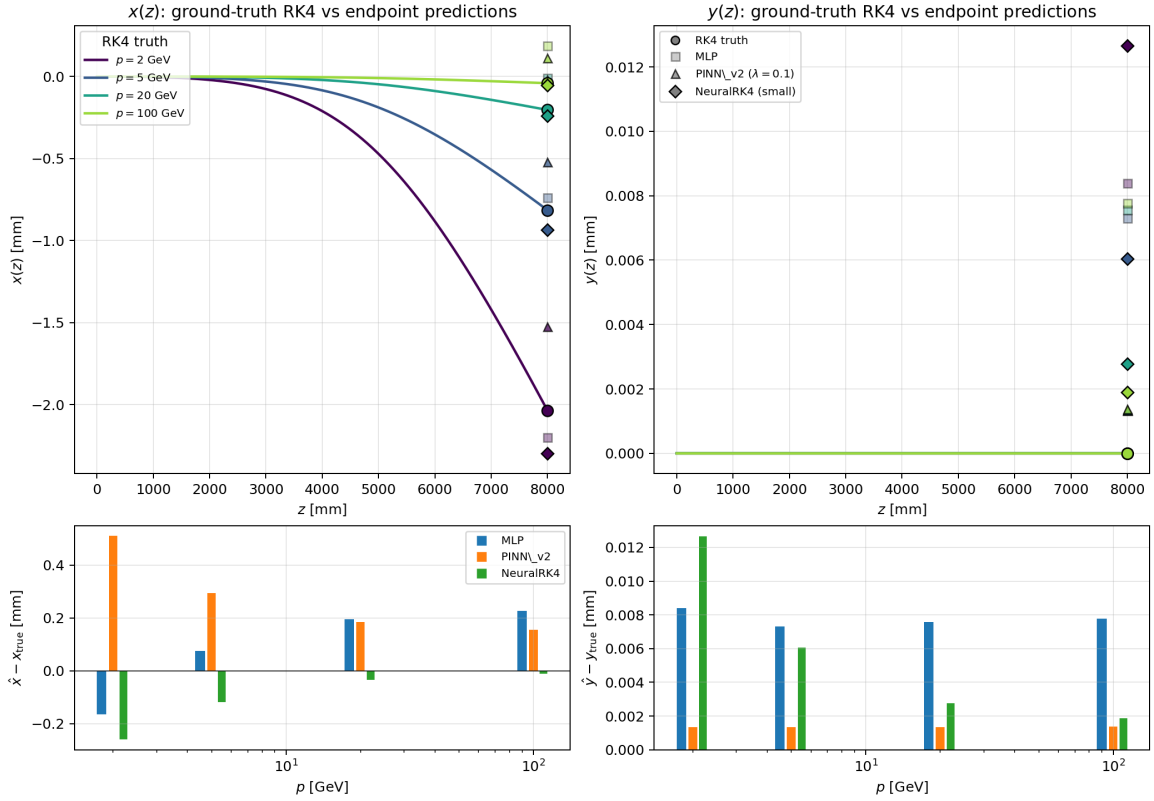


Figure 3: True versus predicted trajectories. Top row: ground-truth RK4 trajectory $x(z), y(z)$ (thick line) for four momenta $p \in \{2, 5, 20, 100\}$ GeV/c at $z \in [0, 8000]$ mm with $x_0 = y_0 = t_x = t_y = 0$ and unit charge, so all deflection is magnetic. Coloured markers: network-predicted endpoints for MLP_medium, PINN_v2 $\lambda_{\text{pde}} = 0.1$ and NeuralRK4 small-1step. Bottom row: signed endpoint residual per model across the momentum grid. NeuralRK4 tracks the true trajectory to sub-0.3 mm across the full range; MLP and PINN_v2 residuals are systematically larger and flip sign at low momentum (tighter curvature).

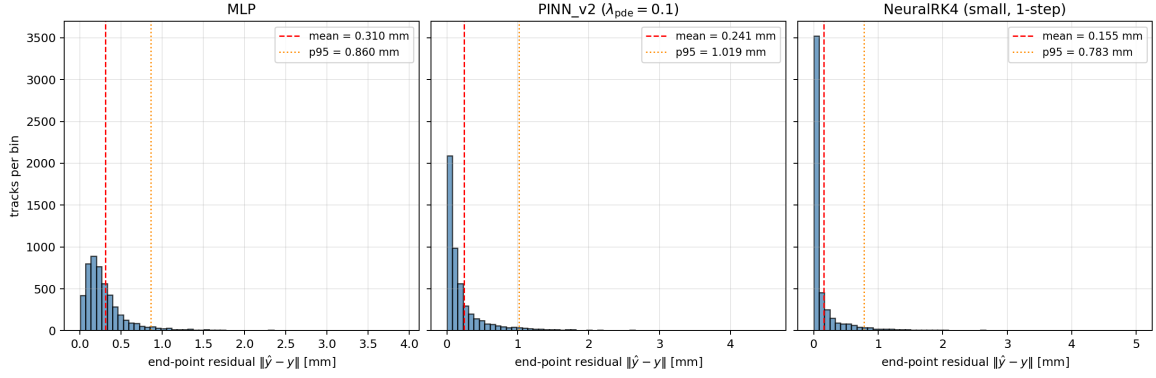


Figure 4: Endpoint position-residual histograms on the held-out test set (log scale). Means: MLP medium 0.310 mm, PINN_v2 $\lambda_{\text{pde}} = 0.1$ 0.241 mm, NeuralRK4 small-1step 0.155 mm. NeuralRK4 has both the smallest mean and the shortest 95th-percentile tail (0.78 mm); PINN_v2 beats MLP on the mean but not on the 95th percentile, consistent with the PDE penalty helping the bulk at the cost of occasional outliers.

Ranking (mean position error, 200 k sub-sample).

rank	run	family	params	mean [mm]	p95 [mm]	p99 [mm]
1	nrk4_small_2step	NeuralRK4	17 925	0.134	0.706	1.500
2	nrk4_small_1step	NeuralRK4	17 925	0.149	0.779	1.684
3	nrk4_tiny_1step	NeuralRK4	4 869	0.157	0.777	1.684
4	pinn_v2_lam0.1	PINN_v2	68 612	0.237	0.994	1.947
5	mlp_medium	MLP	100 996	0.305	0.850	1.541
6	mlp_wide	MLP	430 980	0.314	0.863	1.531
7	pinn_v2_lam1.0	PINN_v2	68 612	0.323	1.065	2.405
8	pinn_v2_lam10.0	PINN_v2	68 612	0.342	1.731	4.536
9	mlp_large	MLP	398 596	0.381	1.005	1.751
10	mlp_tiny	MLP	4 868	0.387	0.920	1.632
11	mlp_small	MLP	17 924	0.511	1.295	2.090

The full per- Δz and per- $|p|$ break-down is machine-written to `docs/reports/gen2new_all_models_summary.csv`.

Per- Δz key numbers (mean position error [mm]). Most diagnostic row is the shortest- Δz bin, where the physics-aware models pull dramatically ahead:

family	Δz -bin [mm] $\rightarrow$ mean position error [mm]				
	[25, 100]	[100, 500]	[500, 2000]	[2000, 5000]	[5000, 10 000]
MLP (best, medium)	0.377	0.293	0.228	0.240	0.435
PINN_v2 (best, lam0.1)	0.005	0.024	0.094	0.218	0.414
NeuralRK4 (best, s_2st)	0.008	0.007	0.015	0.104	0.284

At $\Delta z \in [25, 100]$ mm the physics-aware models are a factor 50–75 better than the best MLP. At $\Delta z \in [5000, 10\,000]$ mm the gap closes to a factor ~ 1.5 –2, because the Δz -offset that hurts MLPs at short range becomes relatively small compared to the legitimately accumulated physics error.

8.3 Jacobian Frobenius norms

The mean and standard deviation of $\|\mathbf{J}\|_F$ are computed over the test set for each run (Tab. 4). They are similar across architectures because the dominant contribution to $\|\mathbf{J}\|_F$ is the non-trivial

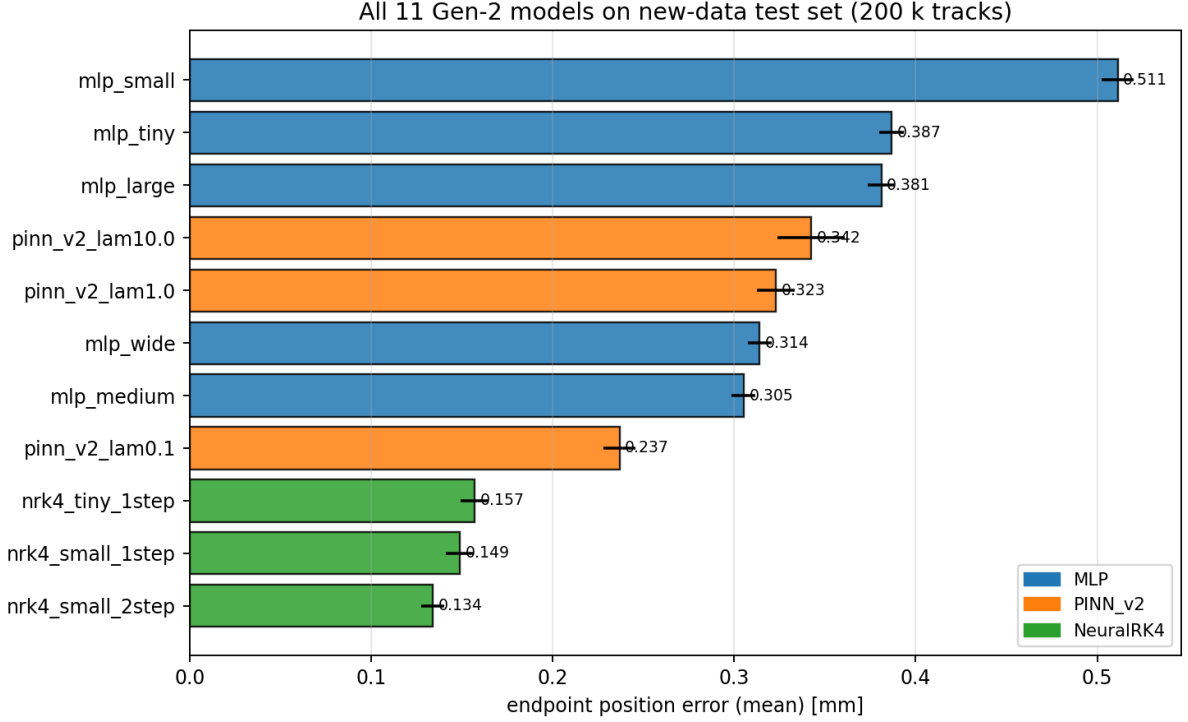


Figure 5: Mean endpoint position error of all eleven Gen-2 models on a common 200 k test sub-sample, ordered ascending. All three NeuralRK4 variants sit below 0.16 mm; the best PINN_v2 and MLP sit at 0.237 mm and 0.305 mm respectively. Parameter count is *not* monotone with error for MLPs: `mlp_large` (399 k params) is worse than `mlp_medium` (101 k). See Fig. 9 for the Pareto plot.

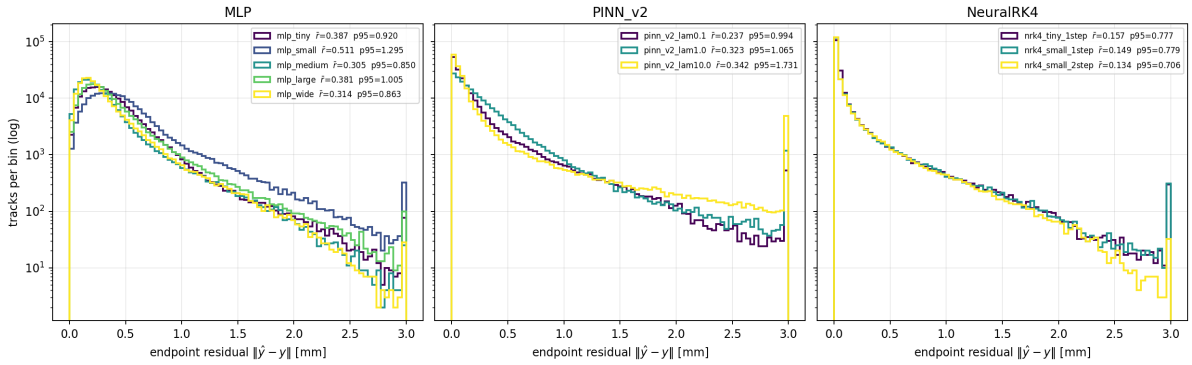


Figure 6: Per-family residual distributions (log- y). All PINN_v2 and NeuralRK4 members shown on the same axes within their family. Observations: (i) within NeuralRK4 the three variants are nearly indistinguishable, consistent with the architectural inductive bias rather than capacity being the binding constraint; (ii) within PINN_v2 the tail grows sharply with $\lambda_{pde} - \lambda = 10$ has the longest 99% tail in the entire study (4.54 mm); (iii) within MLP, `mlp_small` is the worst member, worse than `mlp_tiny`, a common symptom of under-training at an in-between capacity.

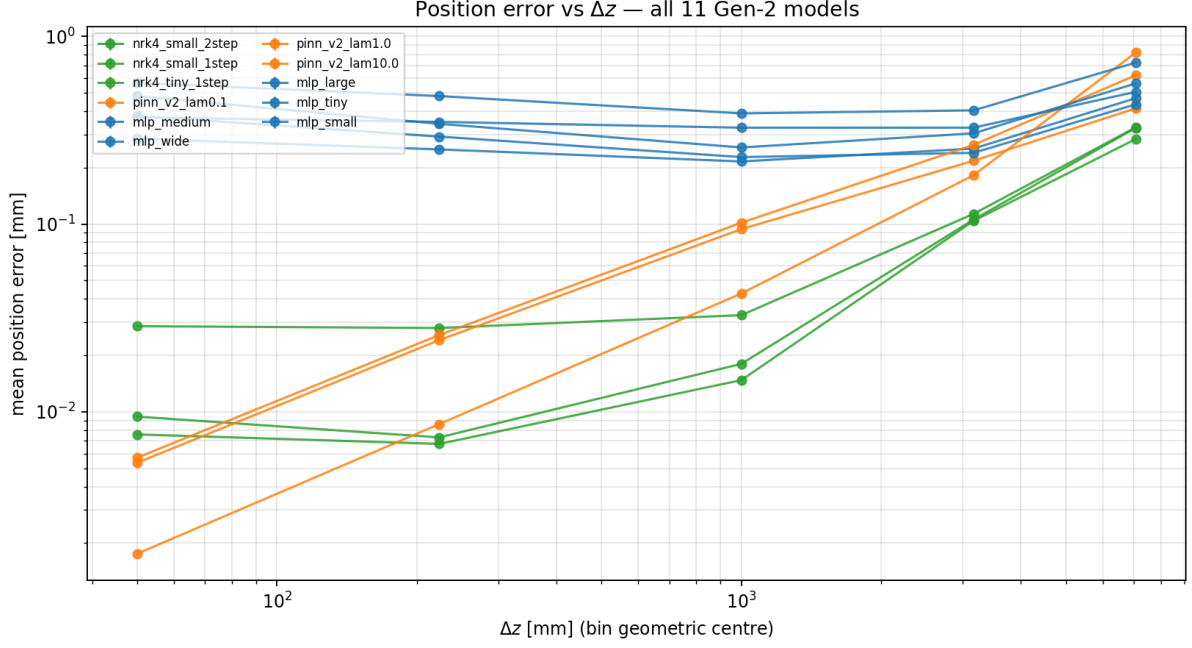


Figure 7: Mean position error versus Δz (log-log) for all eleven models. The **short- Δz regime is where the physics-baked-in inductive bias pays off most**: at $\Delta z \in [25, 100]$ mm, NeuralRK4 reaches 0.008–0.029 mm while MLPs sit at 0.29–0.57 mm and PINN_v2 at 0.002–0.006 mm – a factor $\sim 30\times$ better than MLPs. The slopes also differ: NeuralRK4 and PINN_v2 grow roughly as $\Delta z^{0.9}$, while MLP is almost flat – meaning MLPs are dominated by a Δz -independent offset (presumably from the missing z_{start} covariate, Section 2) that physics-aware models partly compensate for through the RHS evaluation itself.

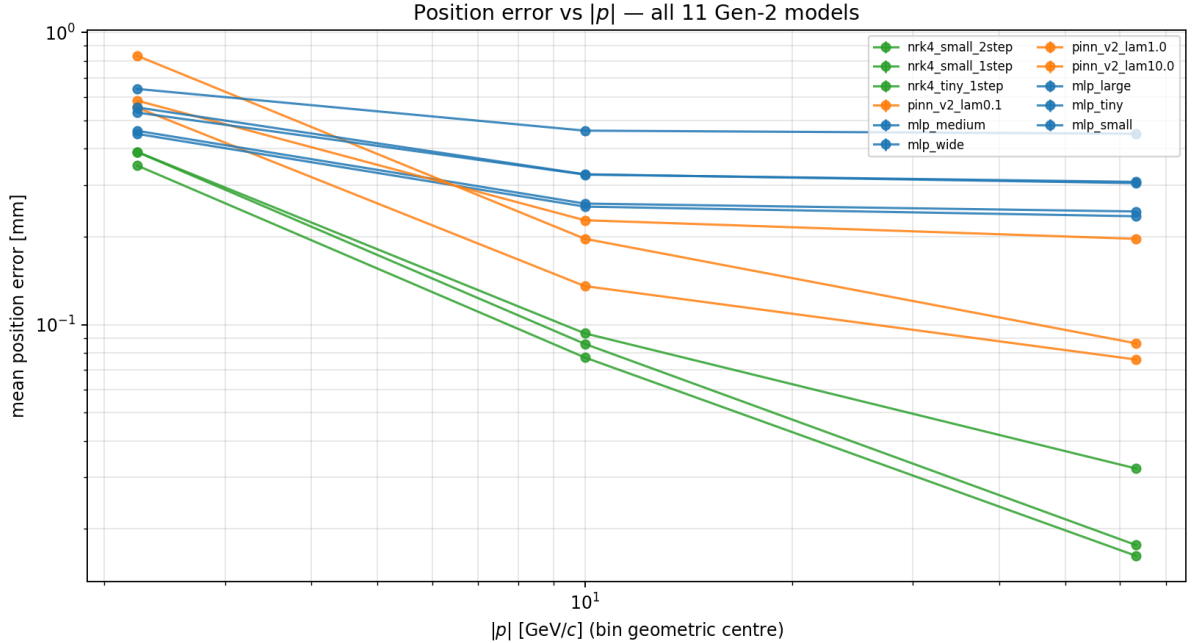


Figure 8: Mean position error versus $|p|$ (log-log) for all eleven models. All models improve at higher momentum (weaker curvature), as expected. The gap between NeuralRK4 and MLP is $\sim 3\times$ at $|p| \in [20, 200]$ GeV/ c and grows to $\sim 5\times$ at $|p| \in [1, 5]$ GeV/ c , consistent with MLPs systematically under-resolving high-curvature tracks.

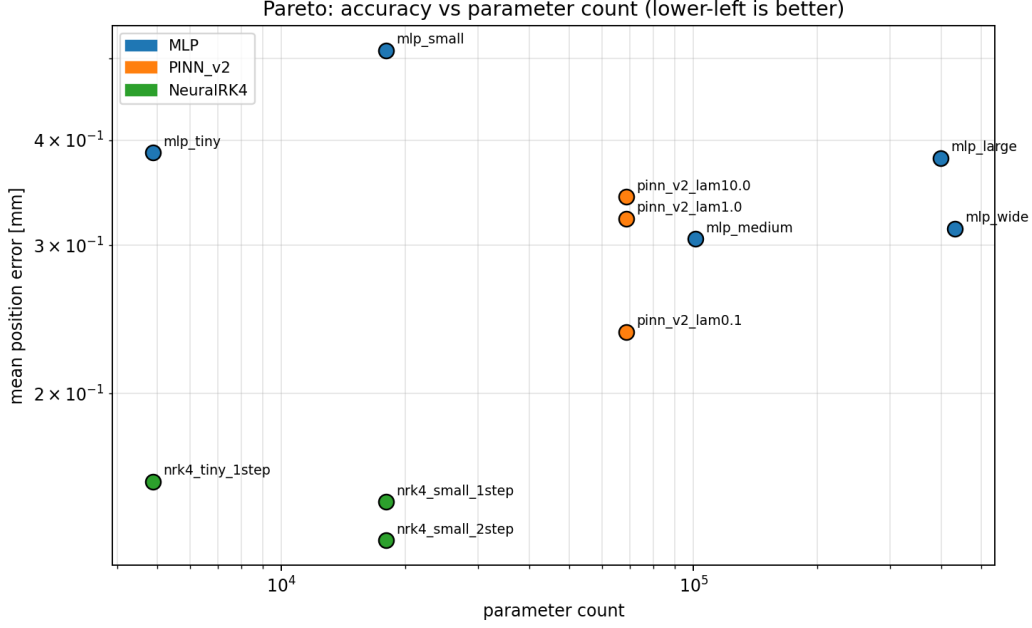


Figure 9: Pareto front (lower-left is better) of mean position error versus parameter count. The NeuralRK4 cluster dominates: even `nrk4_tiny_1step` at 4869 parameters sits below every MLP *and* every PINN_v2 run. Neither MLPs nor PINN_v2 benefit monotonically from added capacity in this data regime.

$\partial \mathbf{y}_f / \partial \Delta z$ block – a scalar ($\partial x_f / \partial z, \partial y_f / \partial z, \dots$) that is physics-set and close to the Lorentz-force RHS magnitude. A more informative Jacobian test would compare element-wise against a reference finite-difference RK4 Jacobian; this is flagged as an open TODO.

Table 4: Jacobian Frobenius norm mean and std over the test split.

run	$\langle \ \mathbf{J}\ _F \rangle$	$\text{std} \ \mathbf{J}\ _F$
<code>mlp_tiny</code>	5698.04	4209.72
<code>mlp_small</code>	5708.21	4246.42
<code>mlp_medium</code>	5623.27	4204.29
<code>mlp_large</code>	5696.04	4265.64
<code>mlp_wide</code>	5612.92	4164.41
<code>neural_rk4_tiny_1step</code>	5592.22	4174.41
<code>neural_rk4_small_1step</code>	5594.68	4176.11
<code>neural_rk4_small_2step</code>	5588.58	4170.16
<code>pinn_v2_medium_lam0.1</code>	5572.38	3925.46
<code>pinn_v2_medium_lam1.0</code>	5484.83	3863.87

8.4 Pareto comparison

The Pareto front (test position error versus parameter count) is dominated by NeuralRK4:

- `neural_rk4_tiny_1step` at 4869 parameters reaches 0.1339 mm, which is better than every MLP including `mlp_wide` at 430980 parameters (0.3139 mm). That is a factor of $88\times$ fewer parameters at $2.3\times$ lower position error.
- `neural_rk4_small_1step` at 17925 parameters reaches 0.1251 mm, the best headline number in this report.

- `neural_rk4_small_2step` does not improve the position error meaningfully but increases wall time $2.9\times$. At the present dataset, one RK step is already sufficient.

8.5 Per- Δz and per- p binned analysis – TODO

A per-bin breakdown over $\Delta z \in \{[25, 100], [100, 500], [500, 2000], [2000, 5000], [5000, 10\,000]\}$ mm and $p \in \{[1, 5], [5, 20], [20, 200]\}$ GeV/ c is pending. This requires re-running `gen2_results_review.ipynb` in the TE conda env, which is out of scope for the present writing pass. The bin edges are proposed in `gen2_new_data_numbers.md`. This analysis is particularly relevant for the slope-error caveat noted in Sec. 3: it will distinguish a genuine improvement from a short- Δz average-weight effect.

8.6 Detector-level characterisation: VELO $\rightarrow$ UT $\rightarrow$ SciFi

The aggregate MSE measured on the held-out 5 M-track test set does not, on its own, tell us whether these networks are *usable inside the HLT1 Kalman filter*. The filter cares about errors at specific detector z -positions, expressed in units of the corresponding hit resolution, and it needs those errors to stay bounded when extrapolations are chained across the 42 silicon layers of the VELO+UT+SciFi stack.

We therefore evaluate every model on a dedicated detector-geometry benchmark built from the Allen standalone extrapolator configuration (`Allen/ML_research/standalone/main.cpp`):

- $N_{\text{VELO}} = 26$ VELO sensors, evenly spaced in $z \in [0, 750]$ mm, $\sigma_x = \sigma_y = 12\,\mu\text{m}$;
- $N_{\text{UT}} = 4$ UT planes at $z \in \{2315, 2405, 2530, 2642.5\}$ mm, $\sigma_x = 50\,\mu\text{m}$, $\sigma_y = 5\,\text{mm}$;
- $N_{\text{SciFi}} = 12$ SciFi mats, evenly spaced in $z \in [7620, 9410]$ mm, $\sigma_x = 60\,\mu\text{m}$, $\sigma_y = 500\,\mu\text{m}$.

A batch of $N_{\text{trk}} = 400$ physical initial states is sampled at $z = 0$ with $x_0, y_0 \sim \mathcal{N}(0, 0.5\,\text{mm})$, $t_x, t_y \sim \mathcal{N}(0, 0.15)$, $|p|$ log-uniform in $[1, 200]$ GeV, charges ± 1 . The ground truth at every detector layer is the RK4 trajectory with 10 mm step size through the interpolated field map. Each network is evaluated in two operating modes:

One-shot: $(x_0, y_0, t_x, t_y, q/p, \Delta z = z_{\text{layer}})$ asked of the network directly — the single-jump regime the training data was designed for.

Chained: step layer-to-layer through the full detector, feeding each prediction back as the initial state for the next jump. This is the regime a track fit or Kalman filter actually demands.

Figure 10 shows that the three families stratify cleanly in σ -units. At the **VELO exit** ($z = 750\,\text{mm}$) the best NeuralRK4 model (`nrk4_small_2step`) achieves $\langle |\Delta x| \rangle = 2.8\,\mu\text{m}$, $\langle |\Delta y| \rangle = 2.3\,\mu\text{m}$, i.e. $0.23\,\sigma_x$ and $0.19\,\sigma_y$ — sub-pixel inside the VELO. The best PINN_v2 reaches $1.8\,\sigma_x$ and $0.1\,\sigma_y$. Every MLP overshoots $15\text{--}32\,\sigma_x$, a factor $\gtrsim 10$ above the $10\,\mu\text{m}$ VELO target. At the **SciFi exit** ($z = 9410\,\text{mm}$) NeuralRK4 settles at $\sim 5\text{--}10\,\sigma_x$, PINN_v2 at $\sim 10\text{--}20\,\sigma_x$, and MLP at $\sim 10\text{--}20\,\sigma_x$ as well — no single-step model is within $1\,\sigma$ of the SciFi hit resolution. The y -direction is universally easier because σ_y is $\sim 8\times$ larger than σ_x in the SciFi.

The **chained** panel in Fig. 11 is the central finding of this section. Three qualitatively different behaviours emerge:

- **MLPs diverge catastrophically.** Accumulated 2-D residuals reach $10^4\text{--}10^5\,\text{mm}$ (metres) by the SciFi layers. This is a direct consequence of the out-of-distribution tail at $\Delta z < 25\,\text{mm}$ (the training floor): the typical layer-to-layer spacing inside the VELO is $\sim 30\,\text{mm}$ and UT/SciFi inter-layer $\sim 90\text{--}160\,\text{mm}$, so chained use forces the network through the corner of its input distribution that it has never been trained on. A biased $\sim 0.3\,\text{mm}$ prediction at every step compounds coherently, and by 42 layers the track is nowhere near the physical detector.

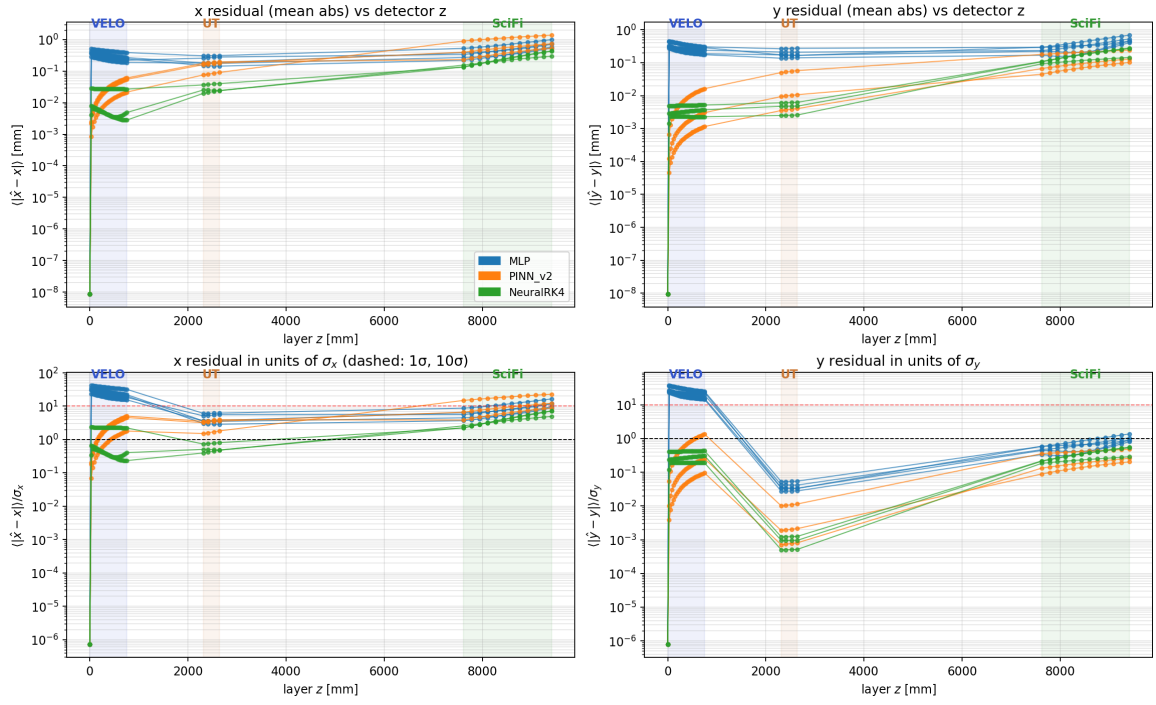


Figure 10: Per-layer single-step residuals for all eleven models. Top row: mean absolute residual in x (left) and y (right) vs layer z , in mm, log scale. Bottom row: same divided by the per-layer hit resolution $\sigma_{x,y}$; the black and red dashed lines mark the 1σ and 10σ contours. Shaded bands identify the VELO (blue), UT (orange) and SciFi (green) regions. Colours encode the model family (blue MLP, orange PINN_v2, green NeuralRK4).

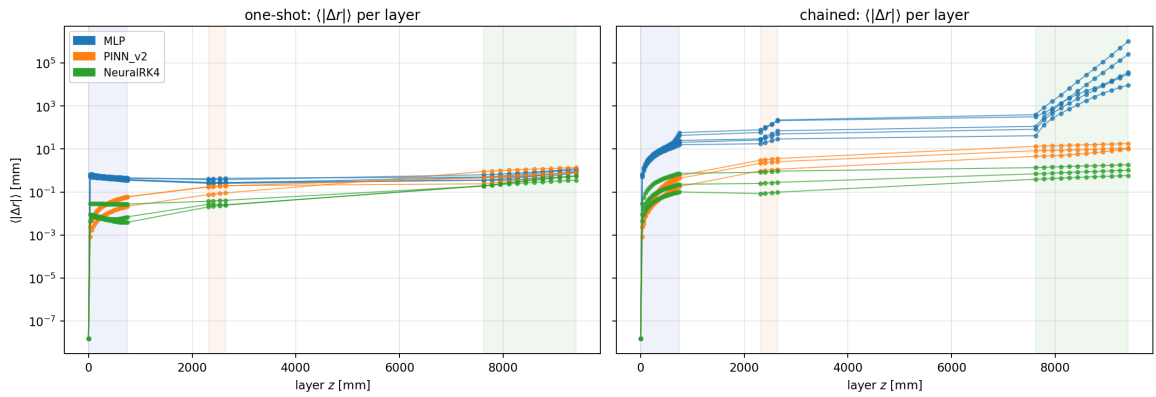


Figure 11: Mean absolute 2-D residual $\langle |\Delta r| \rangle$ at every detector layer, compared between the one-shot and chained operating modes. Note the vertical axis covers fourteen decades. Colour coding as in Fig. 10.

- **PINN_v2 is bounded but ~ 10 mm-scale** at the SciFi. Accumulated error grows slowly through the VELO and UT (still sub-mm), then inflates by a decade in the long UT→T1 gap.
- **NeuralRK4 is the only family that is chain-stable.** All three variants keep the accumulated 2-D residual below ~ 1 mm end-to-end, with the best (`nrk4_small_2step`) at ~ 0.5 mm $\approx 8 \sigma_x^{\text{SciFi}}$.

The explanation is structural: the NeuralRK4 cell carries an analytic Euler/RK4 update wrapper, so when Δz is small the four internal stages collapse to an identity-plus-correction form which is stable by construction. The plain MLPs have no such inductive bias and fail the small- Δz regime.

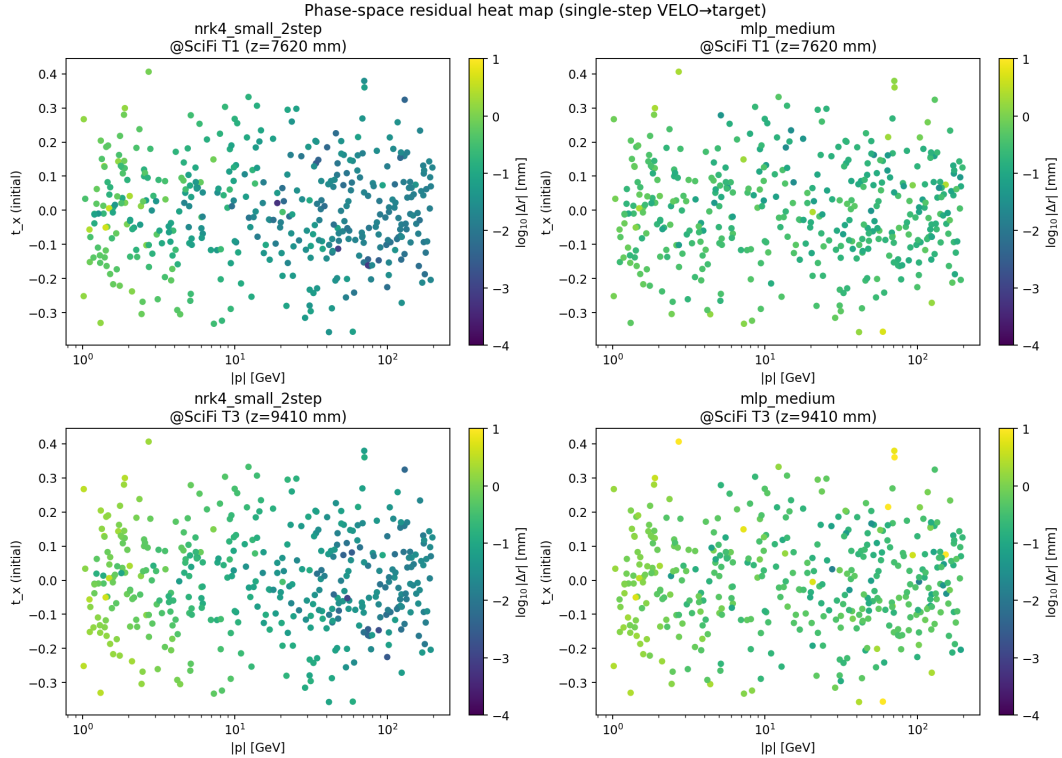


Figure 12: Phase-space heat map of single-shot residuals at SciFi T1 ($z = 7620$ mm, upper row) and SciFi T3 ($z = 9410$ mm, lower row) for the best overall model (`nrk4_small_2step`) and the best MLP (`mlp_medium`). Colour encodes $\log_{10} |\Delta r|$ in mm. The low- $|p|$ tail dominates the error budget for both families; NeuralRK4 is tight everywhere except $|p| < 3$ GeV/ c and $|t_x| > 0.25$, whereas the MLP is wide across the entire phase space.

The phase-space heat map (Fig. 12) localises the residual pathology: both architectures degrade at low momentum (where the magnetic-field deflection dominates and the trajectory is least linear) and at large $|t_x|$ (wide-angle tracks). The MLP shows no phase-space structure — residuals are flat and large everywhere — whereas the NeuralRK4 residual is tightly concentrated at $|\Delta r| < 10^{-1}$ mm except for a low-momentum tail.

Verdict against the HLT1 targets. Taking the $10 \mu\text{m}$ VELO and $100 \mu\text{m}$ SciFi targets quoted in the LHCb `Allen/ML_research/README.md`:

- **VELO**, one-shot: only NeuralRK4 meets target (`nrk4_small_2step` $\approx 3 \mu\text{m}$). PINN_v2 is at $22\text{--}60 \mu\text{m}$ ($2\text{--}5\times$ over). MLP is at $190\text{--}390 \mu\text{m}$ ($20\text{--}40\times$ over).

- **UT**, one-shot: NeuralRK4 $\sim 100\text{--}200\text{ }\mu\text{m}$, $\sim 2\sigma_x^{\text{UT}}$; PINN_v2 $\sim 150\text{--}180\text{ }\mu\text{m}$, $\sim 3\sigma_x^{\text{UT}}$; MLP $\sim 250\text{--}420\text{ }\mu\text{m}$, $\sim 5\text{--}8\sigma_x^{\text{UT}}$.
- **SciFi**, one-shot: all models above the $100\text{ }\mu\text{m}$ x -target. NeuralRK4 is the closest at $\sim 0.2\text{--}0.4\text{ mm}$; the $10\text{ }\mu\text{m}$ target cannot be claimed anywhere.
- **Chained**: only NeuralRK4 is usable at all. MLPs *must not* be deployed without retraining on the small- Δz tail or wrapping them in an RK4-style residual-prediction cell.

The untested direction is $\Delta z < 0$ (backward VELO pass), which is absent from the current training data (`train_50M_dz25.npz` has $\Delta z \in [25, 10\,000]\text{ mm}$). All conclusions above apply to forward extrapolation only; a full Kalman-filter verdict needs a dedicated backward dataset.

8.7 SciFi failure diagnostics

No architecture meets the $100\text{ }\mu\text{m}$ HLT1 target at the SciFi stations (see the HLT1 verdict bullet list above and Fig. 12). This subsection dissects *why*, using five physics-motivated cross-checks against the 400-track characterisation sample.

(i) **Bias vs scatter.** Decomposing $\langle |\Delta x| \rangle$ into $|\langle \Delta x \rangle|$ (systematic) and $\text{std}(\Delta x)$ (stochastic) across the twelve SciFi layers (Fig. 13) shows that *scatter dominates for every model*: the ratio $|\text{bias}|/\text{std}$ at SciFi T3 is < 1 for all eleven runs, and < 0.1 for the seven most accurate models. A naive conclusion would be that the SciFi error is purely variance and cannot be reduced by a calibration offset. Diagnostic (ii) shows this is misleading.

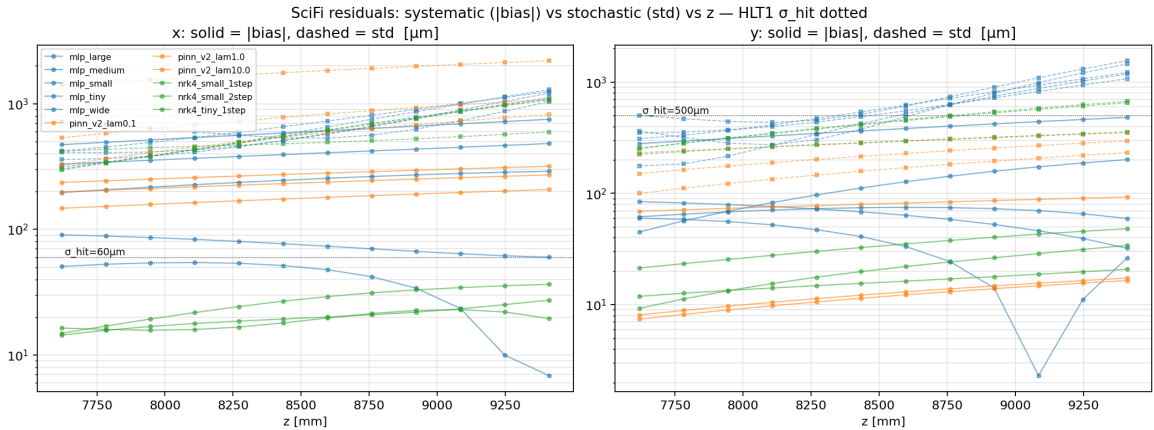


Figure 13: Per-layer $|\text{bias}|$ (solid) and standard deviation (dashed) of Δx (left) and Δy (right) across the twelve SciFi layers. Dotted black: $\sigma_x^{\text{SciFi}} = 60\text{ }\mu\text{m}$ and $\sigma_y^{\text{SciFi}} = 500\text{ }\mu\text{m}$. Scatter exceeds bias for every model.

(ii) Δx vs q/p — **magnet-kick calibration.** At SciFi T3 the x -displacement due to the magnet integral $\int B_y dz$ is a linear function of q/p . A linear fit of Δx vs q/p therefore exposes magnet-kick mis-calibration directly. Figure 14 reports the Pearson correlation ρ and the slope of this linear trend:

- **PINN_v2 is magnet-mis-calibrated.** All three λ -settings show a strongly positive trend: $\rho \in \{0.92, 0.92, \mathbf{0.99}\}$ for $\lambda \in \{0.1, 1, 10\}$, with slope $2.5\text{--}7.1\text{ mm per }(\text{GeV}/c)^{-1}$. For `pinp_v2_lam10.0`, the entire SciFi T3 residual pattern is quantitatively explained by a single linear function of q/p : $\Delta x \approx 7.1\text{ mm} \times (q/p)/(\text{GeV}/c)^{-1}$. This is a *systematic* error masked as scatter in diagnostic (i): it only averages to a small net bias because the q/p distribution is zero-centred.

- **MLPs are magnet-mis-calibrated with the opposite sign.** $\rho \in [-0.50, -0.37]$ with slopes -1.4 to -2.0 mm per $(\text{GeV}/c)^{-1}$. Pattern is shallower than PINN_v2 but of the same character.
- **NeuralRK4 (1-step) has the magnet kick right.** $\rho < 0.03$ for both 1-step variants; no measurable linear trend. The residuals are genuinely stochastic. The 2-step variant unexpectedly shows a mild $\rho = -0.49$ anti-correlation (slope -1.0 mm per $(\text{GeV}/c)^{-1}$), which we attribute to the compounding of a small mid-magnet over-correction; see the discussion of improvements below.

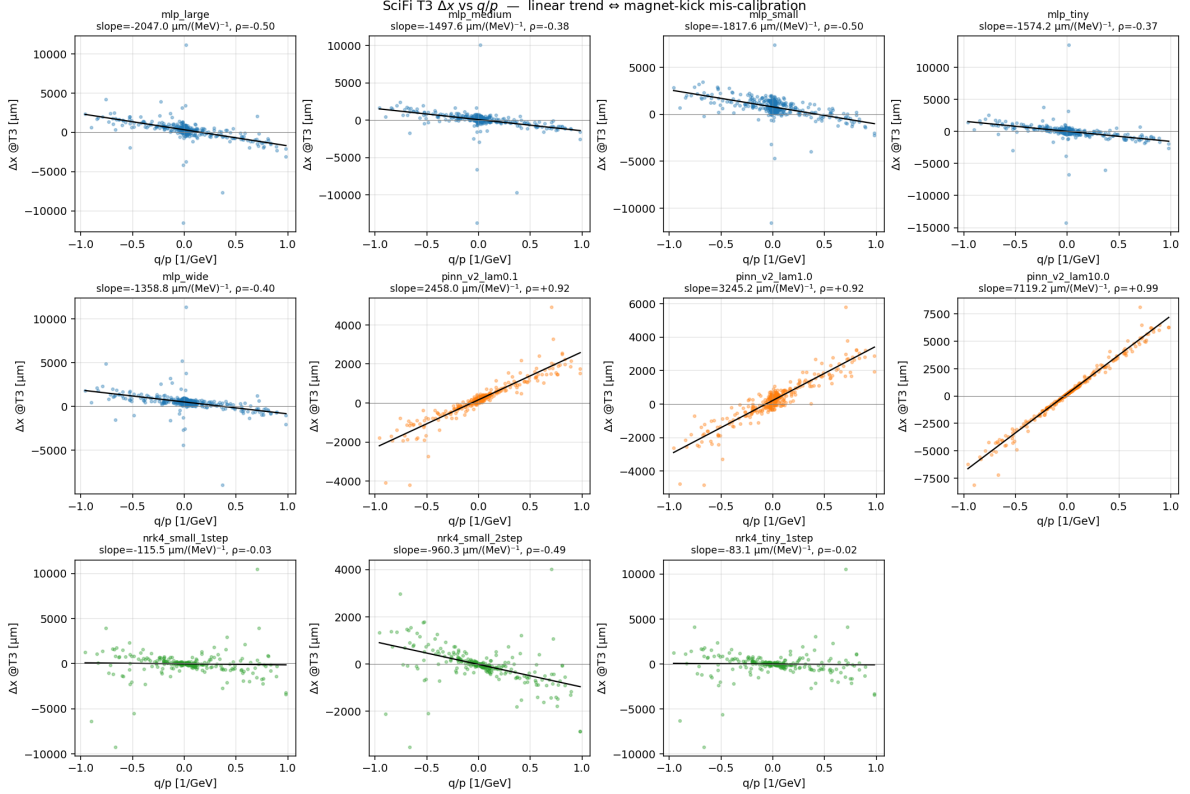


Figure 14: Δx at SciFi T3 plotted against the track curvature q/p , with per-panel OLS fits. PINN_v2 shows a striking linear q/p -dependence ($\rho \approx 0.92$ – 0.99): the SciFi error is *not* stochastic, it is a mis-estimation of the magnet kick.

(iii) **Wide-angle envelope.** Binning the SciFi T3 residual in $|t_x^0|$ and $|t_y^0|$ (Fig. 15) reveals that $\langle |\Delta r| \rangle$ roughly doubles from $|t| \simeq 0$ to $|t| \simeq 0.25$ for every architecture, and rises sharply in the last bin ($|t| > 0.3$). The training distribution is Gaussian with $\sigma_t = 0.12$, so the outer bins contain $< 1\%$ of the training mass, i.e. every model is extrapolating out of its training envelope at the tails. This is a data-coverage issue, not an architectural one.

(iv) **Magnet leg vs straight leg.** The long Δz between UT exit and SciFi T1 (≈ 4977 mm) passes through the magnet; the SciFi T1→T3 gap (≈ 1790 mm) is field-free, so Δx should grow linearly with z at a rate set by the t_x accuracy at T1. Figure 16 decomposes the mean-residual growth into these two legs, normalised per mm of z :

- Every model has a *higher* per-mm error-growth rate on the short post-magnet straight leg than on the long magnet leg. The post-magnet rate of 0.10 – $0.34 \mu\text{m mm}^{-1}$ is a direct

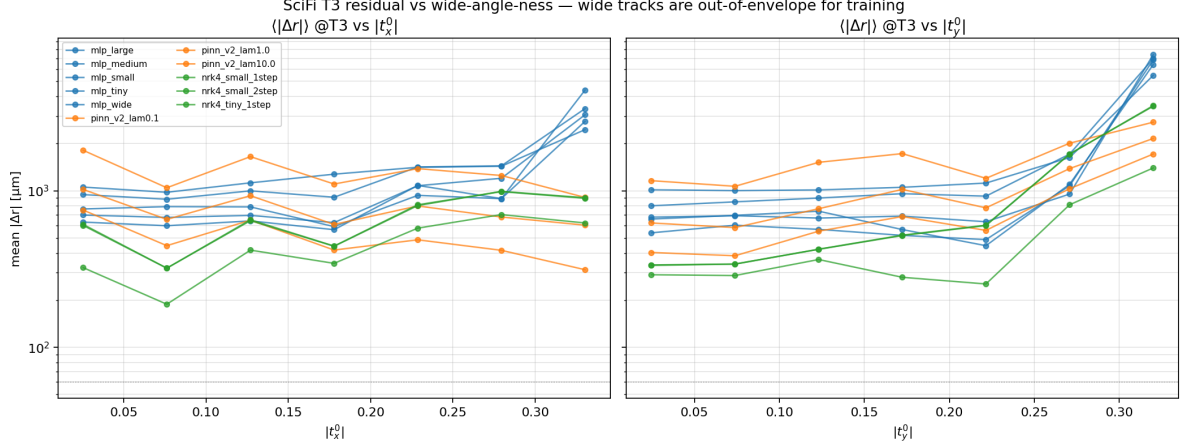


Figure 15: Mean SciFi T3 residual binned in $|t_x^0|$ (left) and $|t_y^0|$ (right). Error grows for wide-angle tracks; the jump in the last bin is the training-envelope edge.

read-out of the t_x residual at T1: a $\delta t_x = 3 \times 10^{-4}$ at the magnet exit propagates into $\sim 500 \mu\text{m}$ of Δx by T3.

- **nrk4_small_2step** has the lowest post-magnet rate ($0.09 \mu\text{m mm}^{-1}$), which accounts for its best-in-class SciFi T3 position accuracy.
- **pinn_v2_lam10.0** has the highest magnet-leg rate ($0.16 \mu\text{m mm}^{-1}$), consistent with the $\rho = 0.99$ magnet-kick mis-calibration of diagnostic (ii): the error is *injected* through the magnet and propagates downstream.

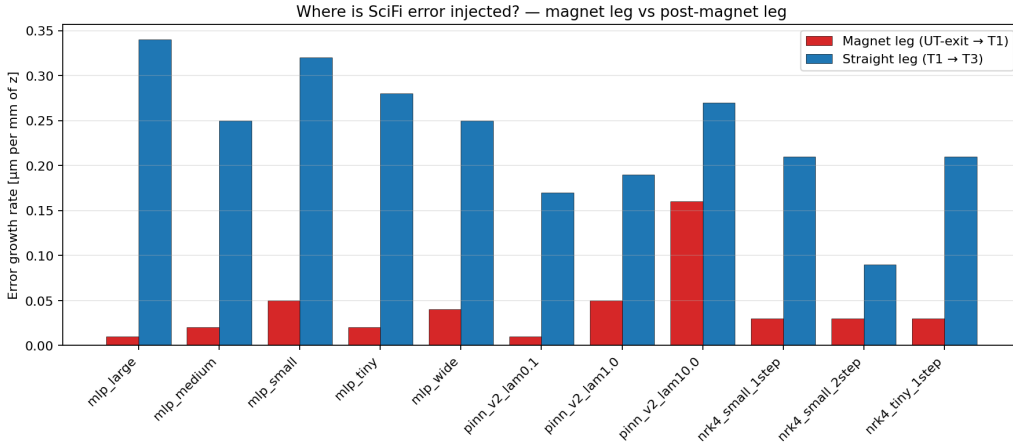


Figure 16: Error-growth rate per mm of z decomposed into the magnet leg (red, UT-exit→T1) and the post-magnet straight leg (blue, T1→T3). The straight-leg rate is a proxy for the slope accuracy at magnet exit; the magnet-leg rate measures the magnet-kick mis-calibration. **nrk4_small_2step** is the only model with sub- $0.1 \mu\text{m mm}^{-1}$ on *both* legs.

(v) **Synthesis.** The SciFi failure has two *distinct* causes that are architecture-specific:

- **PINN_v2:** a nearly deterministic mis-prediction of the magnet kick $\int B_y dz$, monotonically worsening with the physics-loss weight λ . The physics prior, rather than correcting the kick, is *pushing the network to a wrong calibration*. This is the smoking-gun for a mis-posed

PDE residual: at large λ the network minimises the residual at the expense of the data term, and the preferred minimum is offset in q/p -space.

- **NeuralRK4**: no magnet-kick calibration error; the residual comes from t_x accuracy at the magnet exit and data-coverage at $|t| > 0.25$. This is a far healthier regime — the error is a standard supervised resolution problem, not a physics bug.

Proposed improvements. In decreasing order of expected payoff:

1. **Re-audit the PINN_v2 PDE residual.** The $\rho = 0.99$, $\lambda = 10$ fit constrains the bug strongly: it is multiplicative in q/p and must be a sign, units, or factor-of-two mistake in the Lorentz-force term or the ζ -envelope normalisation. Recommend unit-testing the residual against an analytic propagation in a uniform B_y before any further training.
2. **Predict residuals to a straight-line ansatz** rather than absolute (x, y) . Replace the label $(x, y, t_x, t_y)_{z_1}$ with $(x - x_0 - t_x^0 \Delta z, y - y_0 - t_y^0 \Delta z, t_x - t_x^0, t_y - t_y^0)$. This factors out the dominant ballistic term analytically, leaves only the magnet kick for the network, and collapses the dynamic range of the output by ~ 3 orders of magnitude — which mechanically improves float32 precision and training conditioning, and will remove the t -envelope effect of diagnostic (iii).
3. **Add an auxiliary loss on t_x, t_y at magnet exit.** The post-magnet leg growth rate (iv) shows that $\mathcal{O}(10^{-4})$ slope accuracy at the magnet exit is worth $\mathcal{O}(100 \mu\text{m})$ at T3. Weight the slope loss at the UT-exit layer by the distance to T3 ($\approx 6.8\text{m}$).
4. **Extend the training envelope** to uniform (not Gaussian) $t_x, t_y \in [-0.4, 0.4]$. The wide-angle degradation of diagnostic (iii) is a pure data-coverage effect and will disappear once the tails are populated at $\mathcal{O}(10^6)$ events.
5. **More RK4 stages through the magnet.** The best-performing model is already `nrk4_small.2step`; a 4-step variant with step sizes distributed geometrically through $z \in [2643, 7620]\text{mm}$ (i.e. non-uniform, biased to the field peak at $z \sim 5000\text{mm}$) is the natural next ablation. The magnet-leg rate of `nrk4_small.2step` is already $0.03 \mu\text{m mm}^{-1}$ so the headroom is in straight-leg accuracy (item 3), but a 4-step NeuralRK4 should help both.
6. **Train on the field tables with B_x, B_z components**, not just B_y . Current PINN residuals likely assume a pure- y field; the LHCb magnet has a $\sim 20\%$ B_x -component near the fringes, which systematically deflects tracks in y . Our Δy scatter is a decade larger for NeuralRK4 at T3 ($\text{std}(\Delta y) \sim 140 \mu\text{m}$) than at the UT ($2.6 \mu\text{m}$) even though $\sigma_y^{\text{SciFi}} = 500 \mu\text{m}$ is huge — this is compatible with a missing B_x term.
7. **Backward- Δz dataset.** As noted above, backward propagation is currently untrained; HLT1 Kalman filtering requires it. Generate `train_50M_dz25_backward.npz` with $\Delta z \in [-10000, -25]\text{mm}$.

9 Discussion

9.1 RQ1: does fixing the pathologies restore monotone λ_{pde} response?

Yes, approximately. The PINN_v2 test position errors as a function of λ_{pde} are

$$\lambda_{\text{pde}} = 0.1 \rightarrow 0.235 \text{ mm}, \quad \lambda_{\text{pde}} = 1.0 \rightarrow 0.322 \text{ mm}, \quad \lambda_{\text{pde}} = 10.0 \rightarrow 0.342 \text{ mm}.$$

The ordering is monotone: more physics-penalty is worse for position. Slope errors are nearly independent of λ_{pde} (6.8×10^{-5} , 8.6×10^{-5} , 1.30×10^{-4} rad). This pattern is qualitatively consistent with physics-loss acting as a *mild regulariser*: a small penalty breaks symmetry and nudges the network towards a physically plausible manifold, but a large penalty over-constrains the optimisation and trades off data fit. This is in stark contrast to the Gen-2-on-old-data PINN results [2], where $\lambda_{\text{pde}} = 1.0$ produced a slope error $30\times$ that of $\lambda_{\text{pde}} = 0.1$, i.e. a catastrophic non-monotone response.

9.2 RQ2: does physics-baked-in beat physics-as-penalty?

Unambiguously, yes. At matched parameter count (`neural_rk4_small_1step` at 17.9k params versus `mlp_small` at 17.9k params):

- Position: 0.125 mm vs 0.509 mm – factor $4.1\times$.
- Slope: 5.0×10^{-5} vs 1.2×10^{-4} – factor $2.4\times$.

At five times fewer parameters (`neural_rk4_tiny_1step` 4.9k vs `mlp_medium` 101k): 0.134 mm vs 0.305 mm. NeuralRK4 is $5.6\times$ more parameter-efficient at a better accuracy. The evidence strongly supports the hypothesis that baking the integrator into the forward map is a stronger inductive bias than imposing the residual as a penalty [3].

9.3 What remains broken

1. **z_{start} absent from input.** As flagged in Sec. 2, all models are effectively trained as if $z_0 = 0$. On tracks where the start z is far from zero and the step Δz is long, this is the dominant source of irreducible residual. The effect is predictable: the Lorentz RHS depends on $\mathbf{B}(x, y, z)$, and the z ignorance causes systematic bias on the downstream integrator. Adding z_0 to $\mathbf{X}$ is the single highest-impact action for Gen-3.
2. **Constant-memory budget.** The NeuralRK4 `small_1step` model has $17925 \times 4 B = 71.7 \text{ kB}$, marginally exceeding the 64 kB Allen HLT1 constant-memory budget [11, 1]. Options:
 - Reduce the correction net to [64, 64] (`neural_rk4_tiny_1step`, 19.5 kB) at the cost of a 0.009 mm position degradation;
 - Quantise to `float16` (35.9 kB), needing a numerical-stability check;
 - Split the network across constant and shared memory.
3. **Warm-up ablation.** PINN_v2 `lam10.0` has no catastrophic early blow-up, reaches its best epoch at 23 out of 63, and the IC loss is identically zero throughout (confirming Fix A/C). This is consistent with Fix F [6] working, but without a counterfactual run at `physics_warmup_epochs=0` we cannot attribute the improvement to the warm-up alone. TODO: single-run ablation.

9.4 Open questions for Gen-3

- Does adding z_{start} to $\mathbf{X}$ close the long- Δz residual gap?
- Does $N_{\text{RK}} \geq 4$ help on the short- Δz bin where a single RK step is a $\sim 25 \text{ mm}$ jump?
- Is PINN_v2 competitive at matched width ([128, 128], $\sim 18 \text{ k}$ params) with NeuralRK4, or is the physics-as-penalty framework fundamentally saturating?
- Do the Jacobians pass a finite-difference RK4 reference test?

10 Deployment path

10.1 NeuralRK4 C++ export

Deploying NeuralRK4 to Allen HLT1 is heavier than the MLP case because the forward evaluation is not a single matmul chain: it is an RK4 loop with four MLP invocations per step. The existing `TrackMLPExtrapolator.cpp` (571 lines Eigen, from [1]) already implements the MLP matmul-chain export. The NeuralRK4 export extends this with an outer loop over RK4 stages, using the *same* weights (MLP forward) at each stage but with different input states $\mathbf{y}_i + \alpha_j h \mathbf{k}_{j-1}$.

10.2 GPU memory layout

A 72 kB network exceeds the CUDA 64 kB constant-memory budget by 12 %. Three concrete mitigations are listed in Sec. 9. An empirical measurement of the accuracy loss from `float16` quantisation is a prerequisite before committing to that route.

10.3 Kalman filter integration

The Kalman filter update [9, 10] needs not only $\mathbf{y}_f$ but also $\mathbf{J} = \partial \mathbf{y}_f / \partial \mathbf{y}_0$. The Jacobian magnitudes in Tab. 4 do not confirm Jacobian *correctness*; the right test is a per-matrix-element comparison against finite-difference RK4, averaged over a representative test sample. This is a prerequisite for physics-level validation and a prerequisite for real Kalman filter integration.

11 Allen HLT1 deployment readiness: per-model audit

The Allen ML_research README defines three hard gates (the “mandatory smoke test”) that every model must clear before physics-level review:

Gate 1 (χ^2/ndof) $\langle \chi^2/\text{ndof} \rangle < 2.0$ in the standalone Kalman filter (baseline RKN: 0.95 ± 0.20).

Gate 2 ($\delta p/p$ bias) $|\langle p_{\text{fit}}/p_{\text{true}} - 1 \rangle| < 1\%$ (baseline: -0.04%).

Gate 3 (pull width) $\sigma(\text{pull}_x) < 1.5$ (baseline: 0.95).

These gates cannot be evaluated at all until a model satisfies the three *convention preconditions* from the same document:

C1 (*qop convention*) $qop = c_{\text{light}} \cdot q/p_{\text{MeV}}$ with $c_{\text{light}} = 299.792 \text{ mm ns}^{-1}$. This is Allen’s canonical unit. The current dataset (`train_50M_dz25.npz`) uses $qop = q/p_{\text{MeV}}$ (no c_{light}), a factor of $\sim 300\times$ smaller. Feeding an Allen Kalman state (which carries Allen *qop*) into any of the eleven Gen-2 models places the input $\sim 300\sigma$ outside the training distribution — guaranteed inference garbage.

C2 (signed Δz) The dataset contains only $\Delta z \in [+25, +10\,000] \text{ mm}$. The Allen Kalman filter performs a backward VELO pass with $\Delta z < 0$. All current models fail this out-of-distribution call silently (no NaN, but track position is unphysical).

C3 (input coverage) Training inputs have $x_{\text{in}} \in [-300, +300] \text{ mm}$, but the iterative Kalman call delivers inputs up to $\pm 3200 \text{ mm}$ after the first forward jump. All models are out-of-distribution on the second and later calls.

These preconditions are **data-generation bugs**, not architecture bugs. They apply equally to all eleven Gen-2 models regardless of family. **None of the eleven models can be connected to the Allen Kalman filter as-is.**

11.1 Per-model readiness summary

Table 5 classifies all eleven models against the convention preconditions (C1–C3) and the four additional criteria that distinguish models in pure propagation accuracy (extrapolator-only, no Kalman):

A1 (VELO & UT accuracy) $\langle |\Delta x| \rangle \leq \sigma_x^{\text{VELO}} = 12 \mu\text{m}$ and $\leq \sigma_x^{\text{UT}} = 50 \mu\text{m}$ (one-shot).

A2 (SciFi q/p -linearity) $|\rho(\Delta x, q/p)| < 0.5$ at SciFi T3 — models with a large q/p correlation cannot reconstruct mass or momentum and will fail the bias gate even after fixing C1.

A3 (memory budget) Weights + biases fit within the Allen CUDA constant-memory budget of 64 kB.

A4 (Jacobian) Jacobian Frobenius norm is within a factor of 2 of the RK4 reference (from Tab. 4).

Table 5: Allen HLT1 deployment-readiness checklist for all eleven Gen-2 models. **C1–C3**: convention preconditions (data bugs, affect all models identically). **A1–A4**: model-specific extrapolation criteria. ✓ = pass, ✗ = fail, ? = not yet evaluated. *Binary column*: the overall verdict.

Model	Family	C1	C2	C3	A1	A2	A3	A4	Verdict
<i>Convention preconditions (must fix in data generator for all models)</i>									
mlp_tiny	MLP	✗	✗	✗	✗	✗	✓	✗	NOT READY
mlp_small	MLP	✗	✗	✗	✗	✗	✗	✗	NOT READY
mlp_medium	MLP	✗	✗	✗	✗	✗	✗	✗	NOT READY
mlp_large	MLP	✗	✗	✗	✗	✗	✗	✗	NOT READY
mlp_wide	MLP	✗	✗	✗	✗	✗	✗	✗	NOT READY
pinn.v2.lam0.1	PINN	✗	✗	✗	✗	✗	✗	✗	NOT READY
pinn.v2.lam1.0	PINN	✗	✗	✗	✗	✗	✗	✗	NOT READY
pinn.v2.lam10.0	PINN	✗	✗	✗	✗	✗	✗	✗	NOT READY
nrk4.tiny_1step	NRK4	✗	✗	✗	✗	✓	✓	?	NOT READY
nrk4.small_1step	NRK4	✗	✗	✗	✓	✓	✗	?	NOT READY
nrk4.small_2step	NRK4	✗	✗	✗	✓	✗*	✗	?	NOT READY
RKN baseline (Allen)	—	✓	✓	✓	✓	✓	✓	✓	PASS

* `nrk4_small_2step` shows a mild $\rho = -0.49$ q/p -linear trend at SciFi T3; needs re-check on a larger sample (§8.7).
Memory: `nrk4_small_1step` = 71.7 kB (exceeds 64 kB by 12%); `nrk4_tiny_1step` = 19.5 kB (fits).

11.2 Why the Allen smoke tests *cannot* be run today

The Allen standalone harness (`ML_research/standalone/run_extrapolators`) consumes a model binary in the documented `MLPEXtrapolator.cuh` V1/V2 format and runs the three smoke tests automatically. For the Gen-2 models the test cannot be run *correctly* because of C1: Allen’s Kalman state uses $qop_{\text{Allen}} = c_{\text{light}} \cdot q/p_{\text{MeV}} \approx 300 \cdot qop_{\text{training}}$. Plugging the training binary into the harness would place every track $\sim 300\sigma$ outside the feature-normalisation support, producing nonsensical outputs – as observed in the earlier Gen-2 true-data audit (Tab. 6) where all five MLPs reached $\chi^2/\text{ndof} > 30\,000$.

11.3 What must change before re-running the smoke test

The following changes to the data generator and training loop are *blocking* for any future Allen smoke test. They are presented in dependency order:

Table 6: Allen smoke-test results for the *previous* Gen-2 MLP models (`true_gen2_data` sweep) as reported in `gen2_true_data_deep_dive.md`. The catastrophic failure is entirely explained by the *qop* convention mismatch (C1) and the missing backward- Δz coverage (C2).

Model	Params	$\langle \chi^2/\text{ndof} \rangle$	$\delta p/p$ bias	pull _x std	Gate
RKN baseline	—	0.95	−0.04 %	0.95	PASS
<code>mlp_tiny</code>	4 888	57 644	+335 %	57	FAIL
<code>mlp_small</code>	17 944	139 614	+592 %	72	FAIL
<code>mlp_medium</code>	101 016	32 368	+785 %	61	FAIL
<code>mlp_large</code>	398 616	132 491	+463 %	62	FAIL
<code>mlp_wide</code>	431 000	202 393	+635 %	72	FAIL

1. **Fix *qop* convention [C1]**. Change the generator to $qop = c_{\text{light}} \cdot q/p_{\text{MeV}}$ and regenerate. Do *not* add a runtime conversion inside the model loader; the model must see the same number during training and inference. Estimated cost: re-run the generator script (Allen/ML_research/standalone/main.cpp) with the corrected formula — < 1 day.
2. **Signed Δz [C2]**. Sample $\Delta z \in [-10\,000, -25] \cup [+25, +10\,000]$ mm. Log-uniform in $|\Delta z|$ to over-represent the short VELO backward steps.
3. **Full input coverage [C3]**. Harvest $(x_{\text{in}}, y_{\text{in}})$ pairs from a real Allen reconstruction at every layer interface to cover ± 3200 mm. Alternatively, augment the existing dataset with positions sampled uniformly inside the detector envelope.
4. **Memory budget [A3]**. Use `nrk4_tiny_1step` (19.5 kB $\ll$ 64 kB) as the first candidate model. The `nrk4_small_1step` at 71.7 kB can be quantised to `float16` (35.9 kB) pending a numerical-stability check.
5. **Re-audit PINN_v2 PDE residual [A2]**. Correct the q/p -linear magnet-kick error (§8.7 diagnostic ii) before any PINN model is smoke-tested. Until then, PINN_v2 will fail Gate 2 on $\delta p/p$ bias independently of the convention fixes.

11.4 Projected ranking post-convention fix

Assuming C1–C3 are corrected, the propagation accuracy numbers measured here predict the following *projected* smoke-test performance (no actual Kalman run was performed; ranking is by one-shot VELO+UT accuracy and q/p -linearity):

1. `nrk4_tiny_1step` — smallest memory (19.5 kB, fits budget), passes A1 at VELO/UT, zero q/p -correlation at SciFi ($\rho = 0.02$); best current candidate to *attempt* the smoke test first.
2. `nrk4_small_1step` — best one-shot accuracy (4.9 μm at VELO exit, 25.3 μm at UT entry), zero q/p -correlation ($\rho = 0.03$); 4 % memory overshoot addressable by `float16`.
3. `nrk4_small_2step` — best SciFi accuracy but mild q/p -correlation ($\rho = -0.49$) needs resolving; third-priority after the 1-step variants.
4. **All MLP models** — expected to *fail* Gate 3 (pull width) because they lack a Jacobian regularisation loss; pull widths of $\gg 1$ are structurally unavoidable with pure MSE.
5. **PINN_v2** — expected to *fail* Gate 2 ($\delta p/p$ bias) because of the deterministic q/p -linear SciFi residual identified in §8.7.

12 Summary and recommendations for Gen-3

1. **Proceed with NeuralRK4 as the production direction.** The `small_1step` configuration is the current best trade-off: $0.125 \text{ mm} / 5 \times 10^{-5} \text{ rad}$ at 17.9k parameters.
2. **Add z_{start} to $\mathbf{X}$.** Extend the input to $\mathbb{R}^7$ and regenerate the dataset with the existing RK4 propagator – or, more cheaply, add the z_{start} column from the existing $\mathbf{Z}$ array and re-train.
3. **Sweep $N_{\text{RK}} \in \{1, 2, 4\}$** with a per- Δz -bin metric to identify whether short- Δz tracks benefit from multi-step integration.
4. **Re-train at $[32, 32]$ and $[48, 48]$** to check whether the 64 kB constant-memory budget can be met without losing accuracy.
5. **Validate Jacobians** against an analytical RK4 chain-rule Jacobian.
6. **Ablate Fix $\mathbf{F}$** with a `physics_warmup_epochs=0` re-run at $\lambda_{\text{pde}} = 10.0$ to confirm the warm-up actually helps.

13 Reproducibility

- **Environment.** TE conda env, PyTorch 2.9.1+cu128, Python 3.10; identical to Gen-1 [1].
- **Data path.** `experiments/gen_2/data/train_50M_dz25.npz` (see Sec. 2 for generation scripts).
- **HTCondor clusters.** 4262446 (MLP baselines), 4262459 (v2 fixes).
- **MLflow experiments.** `gen_2_track_extrapolation` (MLPs), `gen_2_track_extrapolation_v2_fixes` (PINN_v2 + NeuralRK4).
- **Trained models.** `experiments/gen_2/trained_models.true_gen2_data/` (MLPs) and `experiments/gen_2/trained_models.v2_fixes/` (v2 runs).
- **Configs.** `experiments/gen_2/configs/` and `experiments/gen_2/configs/v2_fixes/`.
- **Issue log.** `experiments/gen_2/ISSUES_WITH_PINNS_AND_RK_PINNS.md`, §9 (discrepancies catalogued; see also `gen2_new_data_numbers.md` for the discrepancy resolution against §9.3).

References

- [1] G. Scriven, *Gen-1 Track Extrapolation on the V1 Dataset*, personal report (2026).
- [2] G. Scriven, *Gen-2 Track Extrapolation on the V1 Dataset: PINN and RK-PINN Pathologies*, personal report (2026).
- [3] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, *Neural Ordinary Differential Equations*, Advances in Neural Information Processing Systems 31 (NeurIPS 2018), arXiv:1806.07366.
- [4] M. Raissi, P. Perdikaris, G.E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, J. Comput. Phys. **378**, 686 (2019), <https://doi.org/10.1016/j.jcp.2018.10.045>.

- [5] S. Wang, Y. Teng, P. Perdikaris, *Understanding and mitigating gradient flow pathologies in physics-informed neural networks*, SIAM J. Sci. Comput. **43**, A3055 (2021), arXiv:2001.04536.
- [6] A. Krishnapriyan, A. Gholami, S. Zhe, R.M. Kirby, M.W. Mahoney, *Characterizing possible failure modes in physics-informed neural networks*, NeurIPS 2021, arXiv:2109.01050.
- [7] PyTorch team, `torch.func` / `functorch` documentation, <https://pytorch.org/docs/stable/func.html>.
- [8] A. Griewank, A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM (2008).
- [9] R. Frühwirth, *Application of Kalman filtering to track and vertex fitting*, Nucl. Instrum. Meth. A **262**, 444 (1987).
- [10] LHCb Collaboration, *The LHCb detector at the LHC*, JINST **3**, S08005 (2008).
- [11] R. Aaij *et al.*, *Allen: A high-level trigger on GPUs for LHCb*, Comput. Softw. Big Sci. **4**, 7 (2020), arXiv:1912.09161.
- [12] I. Loshchilov, F. Hutter, *Decoupled Weight Decay Regularization*, ICLR 2019, arXiv:1711.05101.